

#Python Script

```
import boto3
```

```
import time
```

```
# --- CONFIGURATION ---
```

```
ALB_NAME = 'your-alb-name'
```

```
REQUEST_THRESHOLD = 1000
```

```
WEB_APP_AMI_ID = 'ami-xxxxxxx'
```

```
WEB_APP_INSTANCE_TYPE = 't3.micro'
```

```
WEB_APP_SECURITY_GROUP_IDS = ['sg-xxxxxxx']
```

```
WEB_APP_SUBNET_ID = 'subnet-xxxxxxx'
```

```
MONGODB_INSTANCE_ID = 'i-xxxxxxxxxxxxxx'
```

```
OPENSEARCH_DOMAIN = 'your-opensearch-domain'
```

```
ec2 = boto3.client('ec2')
```

```
cloudwatch = boto3.client('cloudwatch')
```

```
opensearch = boto3.client('opensearch')
```

```
# --- GET ALB METRIC ---
```

```
def get_alb_request_count():
```

```
    response = cloudwatch.get_metric_statistics(
```

```
        Namespace='AWS/ApplicationELB',
```

```
        MetricName='RequestCount',
```

```
        Dimensions=[{'Name': 'LoadBalancer', 'Value': f'app/{ALB_NAME}'},
```

```
        StartTime=time.time() - 300,
```

```
        EndTime=time.time(),
```

```
        Period=300,
```

```
        Statistics=['Sum']
```

```
    )
```

```
    datapoints = response['Datapoints']
```

```
    if datapoints:
```

```
    return datapoints[0]['Sum']  
    return 0
```

```
# --- SCALE EC2 WEB APP INSTANCE ---
```

```
def scale_web_app():  
    print("Launching a new web-app EC2 instance...")  
    ec2.run_instances(  
        ImageId=WEB_APP_AMI_ID,  
        InstanceType=WEB_APP_INSTANCE_TYPE,  
        MinCount=1,  
        MaxCount=1,  
        SecurityGroupIds=WEB_APP_SECURITY_GROUP_IDS,  
        SubnetId=WEB_APP_SUBNET_ID,  
        TagSpecifications=[{  
            'ResourceType': 'instance',  
            'Tags': [{'Key': 'Name', 'Value': 'WebAppInstance'}]  
        }]  
    )  
    print("New web-app instance launched.")
```

```
# --- SCALE OPENSEARCH ---
```

```
def scale_opensearch():  
    print("Scaling OpenSearch domain...")  
    response = opensearch.update_domain_config(  
        DomainName=OPENSEARCH_DOMAIN,  
        ClusterConfig={'InstanceCount': 3} # You can make this dynamic  
    )  
    print("OpenSearch node count updated.")
```

```
# --- UPGRADE MONGODB EC2 INSTANCE ---
```

```
def upgrade_mongodb_instance():
```

```

print("Stopping MongoDB instance...")
ec2.stop_instances(InstanceIds=[MONGODB_INSTANCE_ID])
waiter = ec2.get_waiter('instance_stopped')
waiter.wait(InstanceIds=[MONGODB_INSTANCE_ID])

print("Modifying instance type...")
ec2.modify_instance_attribute(
    InstanceId=MONGODB_INSTANCE_ID,
    InstanceType={'Value': 'm4.xlarge'}
)

print("Starting MongoDB instance...")
ec2.start_instances(InstanceIds=[MONGODB_INSTANCE_ID])
print("MongoDB instance upgraded.")

# --- MAIN ---
if __name__ == "__main__":
    request_count = get_alb_request_count()
    print(f"Request count: {request_count}")
    if request_count > REQUEST_THRESHOLD:
        scale_web_app()
        scale_opensearch()
        upgrade_mongodb_instance()
    else:
        print("Request count below threshold. No scaling performed.")

```

Assumptions:

- CloudWatch metric threshold triggers the script (or run it in cron).
- The ALB is already configured.
- IAM role has EC2, CloudWatch, and OpenSearch permissions.

- This script bypasses Auto Scaling Group (ASG), using raw EC2.